The Machine Speed Papers

Evidence Systems

Why Independent Witnesses Emerge at Machine Speed

Jonathan M. Watson
Independent Researcher
jonmwatson@gmail.com

The Machine Speed Papers, No. 3 — Version 1.0 — July 2026

Contents

Chapter 1 — Testimony From an Interested Party 5
Chapter 2 — Why the Kernel Cannot Prove Itself 7
Chapter 3 — Why Logs Are Not Evidence 8
Chapter 4 — The Minimal Witness 10
Chapter 5 — Independent Discovery, Not Invention 12
Chapter 6 — Why the Kernel Cannot Deploy Without This 15
Chapter 7 — The Factory Floor, No Kernel Required 16
Chapter 8 — The Vendor Who Can Prove Their SLA 18
Chapter 9 — When Witnesses Need to Talk to Each Other 19
Chapter 10 — What the Mesh Requires, and Where It Starts 20
Chapter 11 — The Purchasing Agent That Was Denied 22
Chapter 12 — What the Underwriter Actually Asks For 23
Chapter 13 — The Market That Cannot Be Owned 26
Chapter 14 — Evidence as a First-Class Enterprise Data Type 29
Chapter 15 — Objections 32
Chapter 16 — Prior Art and Honest Placement 34

The Machine Speed Papers

Evidence Systems: Why Independent Witnesses Emerge at Machine Speed

Version 2.0 — Final Manuscript

Jonathan.M.Watson
Independent Researcher
jonmwatson@gmail.com
August 01, 2026

Abstract — Revised

An action can be governed correctly and still be unprovable.

This is the gap this paper addresses.

An Executable Governance Kernel (EGK) operating at an execution boundary can evaluate a proposed action, apply a compiled standard, and reach a determination — permit or deny — with total fidelity. It can do this every time, deterministically, before the action commits. None of that is in question here. What is in question is what happens next: the kernel's own record of what it decided is produced by the same system whose behavior that record is meant to prove. A determination that can only be verified by asking the party who made it is not evidence. It is testimony from an interested party, and every institution that has ever priced risk, adjudicated liability, or underwritten a loss has already refused to accept that kind of testimony as sufficient.

This paper makes seven claims.

First, evidence and governance are not the same function wearing different clothes. A mechanism that governs — that permits or denies — is necessarily prior to the action it governs. A mechanism that provides evidence of that governance is necessarily posterior to it. No engineering closes that gap, because it is not a gap of speed. It is a gap of logical order.

Second, this separation has a consequence usually missed: an evidence function that shares an operator with the system it evidences has not solved the problem it exists to solve. It has moved the self-attestation problem one layer down and left it otherwise intact.

Third, an independent witness — a mechanism that only listens and records, that cannot gate, delay, or influence the system it observes, and that has no write path back into it — is not an optional hardening measure. It is the minimum structural requirement for a record to count as evidence rather than testimony.

Fourth, this requirement is not new, not specific to artificial intelligence, and not waiting to be invented. It has been independently discovered, in different vocabularies, by fire insurers who will not accept a homeowner's account of their own smoke detector, by regulators who require third-party inspection of pressure vessels and sprinkler systems, and — in the last several years — by a growing number of independently built cryptographic witnessing systems that converge on the same architecture without having spoken to one another.

Fifth, this same requirement applies with equal force below and above the threshold of machine speed. It can be deployed today, at the pace of an ordinary factory or an ordinary vendor contract, with no autonomous system anywhere in the building — and it already outperforms the log-and-reconstruct model that current insurance and audit practice depends on. It is not a technology waiting for agentic AI to justify its existence.

Sixth, machine-speed execution does not introduce this requirement. It removes the option of ignoring it. A deterministic kernel that reaches a determination with no independent witness present has produced evidence with no more standing than the self-owned log it was meant to replace — which means the witness layer is not a companion to be built alongside such a kernel. It is a precondition the kernel cannot function without, in any deployment where its output will ever be tested.

Seventh, the mechanism that satisfies this requirement cannot be owned by any single commercial party without collapsing the independence it exists to provide. This produces a specific and unusual economic structure: the protocol must remain unowned while the deployment, verification, and interpretation of its output become a genuine and durable commercial category — the same split that let no one own the internet's addressing scheme while a great deal of the internet's economic value was built on top of it.

These claims are tested against existing systems that have already converged on parts of this architecture, against objections that this is merely audit logging with better cryptography, and against an apparent tension between the independence this paper requires and the economic coupling that some existing systems have deliberately built on top of it — a tension this paper resolves by separating four architectural responsibilities that coupled systems tend to collapse into one.

The requirement this paper describes was not designed. It was discovered, the same way the execution boundary was discovered — present, whether or not anyone has yet built the thing that satisfies it.

A note on method and disclosure

This paper draws on a set of independently developed systems that share significant architectural overlap with the case examined most closely in these pages — a system this author had direct access to during its development. That access is disclosed here plainly, and its use in this paper is bounded accordingly: it informed the derivation, giving the author a system whose internal reasoning could be traced firsthand rather than reconstructed from documentation. It is not presented as a case study, not named, and not used as a benchmark against which other systems are measured or found wanting. *The five primitives in Chapter 4 are derived from the witness's purpose alone and do not depend on it.* Where this paper describes design choices made by other, independently built systems, that description is reconstructed from public documentation, and the resulting comparison is treated with the caution that reconstruction deserves.

Where independently developed systems have converged on the same architectural conclusion without contact with one another, that convergence is presented as evidence in

its own right — a signal that a real constraint has been found more than once, not that any one implementation of it is correct. Where this paper identifies a design choice in an existing system that appears to compromise the independence this paper argues is required, that observation is made once, briefly, and left for the reader to weigh — not developed into a case against any named party. The purpose of this paper is to describe a category taking shape, and to be useful to anyone building within it, including those whose present architecture this paper's own framework would ask them to reconsider.

Chapter 1 — Testimony From an Interested Party

A house burns down. The owner says the fire started in the kitchen, an electrical fault behind the stove. The insurer does not pay the claim on that account.

Not because the owner is assumed to be lying. Because the owner cannot be believed structurally, regardless of whether they are lying. The owner has an interest in the outcome, and an account produced by an interested party, however sincere, is not evidence of what happened. It is testimony. Testimony can be true. It can also be self-serving, mistaken, or shaped after the fact by a hundred small acts of memory doing what memory does under stress. The insurer has no way to tell which, from the inside of the owner's account alone, and so the account is set aside as a starting point for investigation rather than accepted as its conclusion.

What replaces it is a small, well-established machinery of independence. A fire marshal, employed by neither the homeowner nor the insurer, examines the scene and produces an origin-and-cause determination. If the loss is large enough, the insurer sends its own forensic engineer, who does the same work independently and is expected to reach the same answer by a different route. Where the two disagree, a third party — sometimes a court, sometimes an appraisal panel — adjudicates between them. None of this is exotic. It is the ordinary operating procedure of an industry that has been pricing fire risk for over three hundred years, and it rests on one unstated rule so old that it no longer needs to be argued: the party whose behavior is in question cannot be the sole source of the record of that behavior.

This paper is about what happens when that rule collides with a world where actions happen faster than an investigator can be sent.

Consider what the fire marshal's investigation actually is, stripped of its particulars. It is a reconstruction. Nothing recorded the fire as it started. No independent party was present at the moment of ignition, watching, with a timestamp. What exists afterward is char patterns, wire residue, burn depth, and a trained person's inference about what sequence of physical events would produce exactly this residue and no other. The investigation is slow — days, sometimes weeks — expensive, and, in a meaningful fraction of cases, contested: two credentialed investigators can look at the same debris and disagree about where a fire started, which is precisely why fire-cause litigation exists as a specialty of its own.[1]

[1] NFPA 921, *Guide for Fire and Explosion Investigations* (National Fire Protection Association), the standard courts apply under *Daubert v. Merrell Dow Pharmaceuticals*, 509 U.S. 579 (1993), to assess reliability

None of this is a flaw in fire investigators. It is the necessary cost of reconstruction in the absence of a continuous record. The entire discipline of forensic origin-and-cause investigation is a workaround for something that was never built: a witness that was actually present, continuously, independent of the parties to the loss, recording as events occurred rather than inferring them afterward from what survived.

That absence has been tolerable for as long as the events being investigated moved at a pace reconstruction could keep up with. A house does not usually catch fire twice in a week. A forensic engineer arriving three days after a single, bounded, physically legible event can still do useful work, because the event left enough residue to read and there is only one of it to examine.

That tolerance does not survive contact with volume or speed. A factory running thousands of process cycles a day, a trading system executing in microseconds, an autonomous agent placing purchase orders continuously across a workday — none of these leave char patterns. Their history exists, if it exists at all, as data controlled entirely by the party whose behavior would need to be investigated if something went wrong. Ask for the record, and the mutable record comes from the same source the fire marshal was created specifically not to trust.

This is the actual gap this paper addresses, and it is worth stating without the softening a first pass tends to apply: it is not a technology gap. The means to solve it — independent, continuous, tamper-evident recording — already exist in fragments, deployed by companies that have never spoken to one another, built for reasons that have nothing to do with artificial intelligence. What is missing is not the mechanism. What is missing is the recognition that this mechanism belongs to a single category, that the category has a name, and that the requirement it satisfies is the same requirement the fire marshal satisfies, expressed at whatever speed the events being witnessed actually occur.

The question this paper asks is narrower than it might first appear, and worth stating precisely before the rest of the argument builds on it.

Not: how do we make AI systems trustworthy?

Not: what does compliance require?

The question is this: what does a record have to be, structurally, before it counts as evidence rather than testimony — and does that requirement change at all as the thing being recorded moves from a house fire, to a factory floor, to a machine executing a financial decision in eleven milliseconds?

The claim this paper defends is that it does not change. The requirement is the same requirement at every speed. What changes is only whether it is physically possible to satisfy it in time to matter — and, as later chapters will show, a determination made correctly at machine speed by a well-built kernel is worth nothing evidentially if the record of that determination was never witnessed by anyone but the system that made it.

---

of fire origin-and-cause testimony. See also *Werth v. Hill-Rom, Inc.*, 856 F. Supp. 2d 1051 (D. Minn. 2012), in which a hospital's own five-consultant, 296-page origin investigation remained one contested account among several in subsequent litigation.

The fire marshal problem does not disappear once the fire burns in eleven milliseconds. It becomes the only problem.

What follows is an attempt to take that requirement seriously enough to derive it properly, rather than simply gesturing at cryptography and calling the gap closed.

Chapter 2 — Why the Kernel Cannot Prove Itself

The kernel is proven. Prior work has already established what it is and how it behaves: deterministic, fast enough to complete before the action it governs becomes irreversible, built to fail in a declared direction when it cannot finish evaluating. This is not a hopeful design goal. It is settled. This chapter does not re-argue. It starts from it.

Here is what the kernel does. It takes a proposed action. It checks that action against a standard. It returns one of two answers: permit, or deny. It does this the same way, every time, given the same inputs. None of that is in dispute.

But a determination that disappears the instant it is made is worthless to anyone who wasn't standing next to the kernel when it happened. So the kernel also writes down what it decided — the action, the standard, the answer, the time. Call that record the evidence.

Here is the problem. The kernel wrote it.

Say it plainly: the kernel runs on infrastructure the operator owns. The evidence sits on that same infrastructure. So the only account of what the kernel decided comes from the party who has every reason to want that account to say something specific. This isn't a rare edge case. It's the default. Every system built without deliberately fixing this has exactly this problem.

A record made by an interested party is not evidence. It's testimony. Chapter 1 already drew this line with a homeowner and a fire, and nothing changes it here. Testimony can be completely true and still be useless as proof, because the question that matters isn't "what does this party say happened." It's "how do we know this account wasn't shaped, edited, or built after the fact to look good." An honest operator and a dishonest one produce identical-looking output: the operator's own database, as it stands today. Nothing in the record tells you which kind of operator you're dealing with.

This cuts against instinct. A kernel that's deterministic, tamper-resistant, and fails safe feels like it should be trusted on its own word. It shouldn't, and here's why: tamper resistance against the agent it governs is not tamper resistance against the operator who owns the kernel and the disk it writes to. Those are two different threats. A kernel can be completely sealed off from the agent it governs and still share every credential, every drive, every admin login with the one party who'd want to edit the record if the kernel's decision turned out to be inconvenient.

State the claim once, flatly, because the rest of this paper stands on it: an evidence function that shares an owner with the system it's supposed to prove has not solved anything. It has moved the same problem down one level, wrapped it in cryptographic language, and left it exactly as broken.

Here's the structural reason this can't be patched. Governance — permit or deny — has to happen before the action commits. If it happens after, it governed nothing. Evidence has to describe a decision that's already fully made. It happens after, always. These are not two points on one timeline that faster hardware someday closes. They sit on opposite sides of the same instant, and no amount of speed moves either one to the other side. A kernel that also proves itself isn't a faster kernel. It's one thing being asked to stand in two places at once. That's not a hard engineering problem. It's not possible.

And here's what that means in practice. If the kernel's evidence lives only where the kernel runs, checking that evidence means trusting the operator's own servers — the exact trust no insurer, no court, no counterparty has ever handed to a single interested party, in any domain, for any loss that mattered. That's why the fire marshal exists. "Just ask the homeowner" was never good enough. A kernel's self-stored record is the homeowner's account with better encryption. Same weakness. Same party with something to gain telling the story.

This doesn't get fixed by a smarter kernel, a longer log, or a cleaner audit trail. All three still have the same owner. The fix isn't more detail. It's a different party doing the recording — one with nothing to gain from the answer, no way to touch the system it's watching, and no shared infrastructure with the party under question.

What that party has to look like, and how easily it stops being that party without anyone noticing, is the next chapter.

Chapter 3 — Why Logs Are Not Evidence

Every system that runs produces some kind of record of itself, a log file. That's not new. That's not the gap. A mutable text file, produced by the system under question, is not the thing that's missing.

The gap is this: a record and a witness are not the same thing. Nobody had to notice the difference for as long as the systems producing records moved slowly enough, and mattered little enough, that it didn't matter which one you had.

A log is a system's own account of itself, kept on its own infrastructure, for its own convenience. It was never built to answer the question this paper is asking. It answers a smaller one: what did this system do, as best it can tell you, assuming nothing went wrong in the telling. A log file is built for a support intervention. Not evidence for liability.

That assumption is buried in how logging works, and it's never stated because it never had to be. A log assumes a human will eventually read it, at an unhurried pace, once some other alarm has already flagged a problem. It assumes event volume low enough for a person to page through. It assumes the system producing the log and the system under investigation are, for practical purposes, one trustworthy thing — that if something went wrong, it went wrong somewhere the log will show, not in the log itself. And it assumes causality is local: the answer to "why did this happen" sits a few lines up, same file, same machine, same narrow window of time.

None of that is true anymore.

A system running thousands of actions a second doesn't produce a log a person can read. It produces volume only another automated system can process — which means the first layer of interpretation is itself unaudited machinery, several steps removed from anyone who could catch a problem. A multi-agent workflow, where one agent's output becomes the next agent's input across several handoffs in seconds, has no local causality — the reason something happened may sit four systems and two organizations upstream of the log entry that recorded it, and no single log was ever positioned to see the whole chain. And centralized ownership — the assumption that a system and its own record of itself can be trusted as a package — is exactly the assumption Chapter 2 already broke.

Call a log file what it is: forensic exhaust. A byproduct of the system having run, generated for its own convenience, in its own custody, read by its own tools. That has real operational value. Nothing here says otherwise. But a byproduct is not a witness. The gap between them is the whole gap between "the system says this happened" and "this happened, and you can check it without asking the system."

The kernel makes this concrete. It's proven — deterministic, fails safe, complete every time. Its log of its own determinations can be detailed to the millisecond: what was proposed, what standard applied, what was decided. None of that changes what the log is. It's the kernel's own account of the kernel's own behavior, stored wherever its operator chose to store it, editable by whoever holds admin access to that location. A dispute over whether the kernel actually reached the determination it claims can't be settled by producing more of this log, in finer detail, with better formatting. It can only be settled by asking someone other than the kernel.

State the line plainly, because everything after this chapter depends on it holding without slippage: *A log can be produced by the system it describes. A witness cannot.*

A witness is not a more careful log. It's not a log with cryptographic signatures bolted on afterward. It's a different kind of thing entirely: a mechanism that watches a system's behavior without being that system, without sharing its infrastructure, and — this is where most existing systems quietly get it wrong — without any way to reach back into the system it's watching. A witness listens. It records what it sees, on its own infrastructure, under its own custody. It does not gate, delay, permit, deny, advise, or influence the thing it watches. Not once, not under any condition. The moment a witness gains any power to act back on what it observes — pause a suspicious transaction, flag it for review, hold a determination pending some check — it has stopped being a witness. It has become a second execution boundary, built by accident, answerable to nobody, without any of the design discipline the first one was built with.

This is the most common way a good system quietly turns back into a log. A team building a monitoring layer adds "just one" intervention — an automatic pause on anomalies, a hook that lets someone halt a transaction before it settles — and has rebuilt the exact interested-party problem this whole structure exists to avoid. One layer removed. Harder to spot.

The fix isn't caution. It's a rule with no exceptions: a witness only listens and records. Anything more, however well-meant, isn't hardening the witness. It's building a second kernel and calling it something else.

What a mechanism has to look like to actually satisfy that rule — and what it costs to get even slightly wrong — is the next chapter.

Chapter 4 — The Minimal Witness

Chapter 3 named the rule: a witness only listens and records. Nothing else. This chapter builds the thing that follows that rule, piece by piece, and stops the instant adding anything else stops being necessary.

Start from the job itself, stripped down. The witness has one purpose: take something that happened, and make it provable to someone who wasn't there, without asking the party it happened to. Everything the witness contains has to earn its place against that one purpose. Nothing else gets in.

Five things turn out to be necessary. No more.

A captured event. Something happened — the kernel reached a determination, a sensor read a value, a state changed. The witness needs a way to take that event in, the moment it occurs, without waiting and without asking permission. If the witness has to request the event, or query for it after the fact, the party being witnessed controls when and whether the witness sees anything at all. That is not capture. That is being told a story on a schedule someone else sets.

A chain. One recorded event proves nothing on its own — it could have been inserted, backdated, or slipped in later to look older than it is. Each event has to be linked to the one before it, so that altering any single record breaks the chain at that exact point and shows it broke. This is not a nice-to-have. An unchained record and a diary entry are the same thing: writable at any time, by anyone with access, with no way to tell when it was actually written.

An external time reference. A timestamp generated by the same clock the witnessed system runs on proves nothing, because that clock answers to the same operator the witness exists to check. The time has to come from outside — a source the operator cannot adjust, backdate, or pause. Without this, "recorded at 9:04am" means "the system says it was 9:04am," which is testimony with a number attached, not evidence.

Administrative separation. The witness's infrastructure — the credentials, the storage, the keys — cannot be reachable by whoever administers the system being witnessed. Not "harder to reach." Not "reachable but logged." Unreachable. If the same person who runs the kernel can also touch the witness's disk, the witness is not independent. It is a second folder on the same desk.

A disclosure interface. None of the above matters if nobody outside the witness can ever check it. There has to be a way for a third party — an auditor, a court, an insurer, a counterparty — to verify the chain, the timestamps, and the content, without needing

the cooperation of the system being witnessed. If verification requires the witnessed party to hand over a key, unlock a database, or explain the format, the witness has quietly become dependent on the same party it was built to check.

Captured event. Chain. External time. Administrative separation. Disclosure interface. Five things. Every one of them fails the whole system if it's missing.

One constraint follows directly from the disclosure interface. It is not a sixth primitive — it is what the fifth one has to do when the witness cannot do its job.

A witness can fail. A key expires. Storage goes unavailable. The external clock disagrees with itself. The chain is incomplete across some window. When that happens, the witness has lost the ability to distinguish a genuine event from a fabricated one, and everyone downstream has lost it too. At that point there are exactly two options: report confidence it does not have, or report that it cannot attest.

The first destroys the only property the witness exists to provide. A mechanism that fabricates certainty once cannot be trusted at any other point either, because nothing in the record distinguishes the fabricated interval from the honest ones — which is precisely the failure Chapter 2 identified in the kernel's self-written log, arriving now one layer further out.

So the second is not a design preference. A witness must never fabricate confidence. When the integrity of evidence cannot itself be established, the witness must disclose its inability to attest, and that disclosure must be captured, chained, and anchored like any other record. A gap the witness declares is evidence. A gap it conceals is the self-attestation problem returning through the back door.

This is worth separating from conventional fault-tolerance vocabulary, because the two neighboring mechanisms fail in opposite directions and the words invite confusion. An execution boundary fails closed: if admissibility cannot be established, the action is denied. A witness does neither, because it has nothing to permit or deny. It fails transparently — it keeps reporting, and what it reports is that no trustworthy attestation can be made for the period in question.

Now the test that matters more than the list itself: what doesn't belong on it.

The obvious temptation is a sixth item — some kind of verification that the captured event is *true*, not just that it was captured and chained correctly. Shouldn't the witness check that the kernel's reported determination actually matches what the kernel did? Shouldn't it confirm the sensor reading wasn't spoofed before recording it?

No. And the reason matters more than the answer.

The witness is not an investigator. It does not establish truth. It establishes that a specific record existed, unaltered, at a specific time. Whether the content of that record was itself accurate — whether the kernel's reported determination reflects what actually happened inside the kernel, whether a sensor was tampered with before its reading ever reached the witness — is a different problem, solved by different means, upstream of the witness entirely. Ask the witness to verify truth instead of custody, and it stops being small. It needs to understand the kernel's internals, or the sensor's calibration,

or a dozen other domain-specific things that have nothing to do with recording. It has become an investigator wearing a witness's name, and an investigator is not disjoint. An investigator has to reach in, look around, and form a judgment — and the moment it does that, it can also be argued with, pressured, or wrong in a way that's someone's fault. A witness cannot be argued with. It only has one job: this exact record existed, unaltered, at this exact time. That claim doesn't need investigating. It needs checking, which any outsider can do without trusting the witness's judgment about anything.

This is the same discipline a court applies to a security camera. Nobody asks the camera whether the person it filmed was telling the truth. The camera didn't investigate anything. It recorded what was in front of it, continuously, on hardware the person being filmed doesn't control, with a timestamp nobody in the frame can edit. That is the entire job. Truth is for the court to work out afterward, using the recording as one piece of evidence among others. The camera that also tried to judge guilt would be a worse camera and a bad judge.

Five primitives. Nothing that investigates. Nothing that judges. Nothing that reaches back into what it watches.

Whether anyone has actually built this, or whether it had to wait for this paper to derive it, is the next chapter.

Chapter 5 — Independent Discovery, Not Invention

The five primitives in the last chapter were not built for this paper. They were derived here, for the first time, from nothing but the job a witness has to do. That is worth stating, because what comes next could easily be mistaken for the opposite: a description of things other people have already built, retrofitted to match a conclusion decided in advance.

It isn't. The primitives came first. What follows is what happens when you go looking for them afterward, in systems built by people who never read this argument and never spoke to each other.

They are already there. Repeatedly.

Guardtime built a hash-aggregation system for Estonian e-government records over a decade ago, with no interest in AI, no interest in agentic execution, and no knowledge that anyone would later derive the same five requirements from first principles. It has an external time reference. It has a chain. It has a disclosure interface good enough to carry legal standing under EU trust-service law.[2] Nobody at Guardtime called this "the

[2]Guardtime's KSI has been accredited as a trust service under the EU eIDAS regulation since 2020. e-Estonia, "The use of the National Blockchain Infrastructure to support the E-Residency Initiative in Estonia," Interoperable Europe Portal.

minimal witness." They built it because a hash-aggregation system that answered to its own clock and its own custody wouldn't have been worth building.[3]

The convergence is not limited to companies building witnessing products. It shows up just as clearly in how the market is grading governance maturity. One assessment framework for AI agent governance states plainly that most governance programs cannot independently prove enforcement occurred, because their policy engines sit inside the same process boundary as the agent they're supposed to govern — and that the highest tier of enforceability requires independent runtime observation from outside that boundary.[4] That is Chapter 3's disjointness argument, reached by an assessment methodology that never used the word "witness" and was never exposed to this paper's derivation. A separate runtime-authorization framework defines its own core requirement almost identically: policy evaluated on the hot path, before a tool call, workflow step, or API write ever leaves what it calls the trusted boundary.[5] Neither framework cites the other. Neither uses this paper's vocabulary. Both landed on the same dividing line Chapter 4 derived from first principles: inside the boundary doesn't count, regardless of how detailed the policy documentation is.

This is also visible at the level of who is now funding infrastructure to solve it. A venture round in mid-2026 backed an agent-authorization platform explicitly built around separating an agent's reasoning from its ability to act, enforcing policy at the moment of execution rather than in advance, and producing an audit record of what was actually authorized.[6] Capital does not fund an architecture unless the market has already started demanding it. That the funded architecture separates reasoning from execution and treats the audit record as a distinct deliverable, not an afterthought, is one more independent party arriving at the same shape this paper derives from the ground up.

The same convergence is visible across the research literature, arriving from teams with no contact and no shared vocabulary. Between February and June 2026 alone, at least four independently published architectures separated the governance function out of the agent entirely — placing it in a dedicated runtime layer positioned before execution, and treating the resulting audit record as a distinct, non-optional artifact rather than a

[3] Guardtime's Keyless Signature Infrastructure (KSI) has operated since 2008; formal cooperation with the Estonian government began in 2011, extending KSI across e-health, e-law, e-court, and other government registries. Guardtime, "Estonian Government, Guardtime Accelerate Adoption of Blockchain Technology to Secure 1M Patient Health Records," press release, March 3, 2016; SDxCentral, "Verizon to Use KSI Blockchain Technology Developed for Estonia," 2018.

[4] ARMO, "AI Agent Governance: From Policy Framework to Runtime Enforcement" (2026). The framework's maturity model holds that governance programs relying on policy engines inside the agent's own process boundary cannot independently demonstrate that enforcement occurred, and places independent runtime observation from outside that boundary at its highest tier of enforceability.

[5] TrigGuard, "Runtime Authorization for AI Agents," trigguardai.com/runtime-authorization-ai-agents (2026): runtime authorization defined as evaluation on the hot path before a tool call, workflow hop, or API write leaves the trusted boundary, returning PERMIT, DENY, or SILENCE prior to any side effect committing.

[6] Arcade.dev, $60 million Series A funding round, June 15, 2026, led by SYN Ventures with participation from Morgan Stanley and Wipro; architecture separates agent authorization from action execution and maintains a complete audit trail of which agent performed which action, on behalf of which user, against which resource. Business Wire, "Arcade Raises $60M to Become the Secure Action Layer Behind Every Production AI Agent," June 15, 2026.

logging convenience.[7] Different institutions, different countries, different arXiv subject classes, months apart. None cites another. What each one reached independently is the first half of this paper's argument: governance that sits inside the thing it governs isn't governance. What none of them reached is the second half — that the audit artifact they each specify remains the operator's own account until something disjoint has touched it.

The convergence now extends to the open-standards side of the same space. An open, royalty-free decision-record standard — placed in the public domain, stewarded by a nonprofit foundation, published with a full public implementation trail, producing its first record in early 2026 — independently arrived at chained capture, cryptographic signing, and sub-microsecond recording at the moment of inference, built for court admissibility under Canadian evidence law rather than for anything agentic.[8] Run through the last chapter's filter, it holds on capture and chain, and stops short of administrative separation and external time — its record deploys on the infrastructure of the party it describes, which is the most common drift and the hardest to see from inside. That observation is made once, here, in the spirit the disclosure note promised. What the standard's licensing choice anticipates — nobody owns it, anybody can build it — is an argument this paper does not reach until Chapter 13, arrived at independently, which is its own kind of confirmation.

This is the pattern worth naming plainly: when independent parties, with no contact and no shared incentive, keep landing on the same architecture, that is not a coincidence to explain away. It is what discovery looks like. Nobody converges by accident on five specific, unglamorous requirements unless those requirements were actually there to be found.

Not every attempt gets there. Some systems marketed as tamper-evident keep their record on the same infrastructure, under the same administrative credentials, as the system being recorded — which means whoever runs the application can also rewrite its own history, and the word "immutable" is doing work the architecture doesn't back up. Some systems put a signature requirement directly on the path of the thing they're

[7] Errico, H. (2026). *Autonomous Action Runtime Management (AARM): A System Specification for Securing AI-Driven Actions at Runtime*. arXiv:2602.09433 [cs.CR], February 10, 2026 — arguing that log aggregation, perimeter defense, and post-hoc forensics cannot protect systems whose actions are irreversible and execute at machine speed. Mazzocchetti, A.M. (2026). *Cryptographic Runtime Governance for Autonomous AI Systems: The Aegis Architecture for Verifiable Policy Enforcement*. arXiv:2603.16938 [cs.CR], March 15, 2026 — specifying an Immutable Logging Kernel as a component distinct from the enforcement kernel. Qin, X., Luan, S., See, J., Boukhers, Z., Yang, C., & Li, Z. (2026). *Harnessing Embodied Agents: Runtime Governance for Policy-Constrained Execution*. arXiv:2604.07833 [cs.RO], v2, April 10, 2026 — externalizing governance into a dedicated runtime layer separate from agent cognition. Kaul, A., Lan, Q., & Gupta, P. (2026). *Agent-Bound: Verifiable Behavioral Governance for Autonomous AI Agents*. arXiv:2606.30970 [cs.AI], June 2026 — a decoupled governance layer detached from agent decision-making, positioned directly before action execution, treating the agent runtime as untrusted.

[8] Genesis Glass Foundation / Fondation Genèse Cristal, *Agentic Genesis Decision Record (AgDR): Open Standard for Court-Admissible AI Decision Records*, accountability.ai (2026). Protocol released under CC0 / Apache 2.0 dual license; reference implementation AgDR-Phoenix published via GitHub, PyPI, and crates.io; BLAKE3 chained records with Ed25519 and optional ML-DSA signatures; sub-microsecond capture benchmarks self-reported; first decision record reported February 18, 2026. Compliance mappings to Canada Evidence Act s.31.1 and EU AI Act published by the Foundation. Foundation federally incorporated under the Canada Not-for-profit Corporations Act, Corp. No. 1779509-2.

watching, which means the witness now adds latency to execution — at which point it has quietly become a second gate, not a witness, whatever the documentation calls it. Some systems never leave their own clock. A witness vouching for its own time is a witness vouching for itself.

None of this is named here as an indictment of the teams who built these things. Most of them were solving a real problem, competently, without the benefit of having this argument in front of them first. The point is narrower: the five primitives in Chapter 4 are not a wish list. They are a filter, and running any existing system through that filter tells you immediately where it holds and where it quietly doesn't — which is exactly the use this paper intends the filter to have.

What matters for the rest of this paper is not which existing system got closest. It's that enough of them got close, independently, to confirm the thing being converged on is real. The next chapters take that as settled and move to what it costs — in liability, in underwriting, in plain commercial terms — when a system gets this filter wrong.

Chapter 6 — Why the Kernel Cannot Deploy Without This

Put the last five chapters together and a sequencing question falls out that nobody asked yet: which comes first, the kernel or the witness?

The instinct is to treat them as a matched pair, built and shipped together. Wrong instinct. They are not peers. One is a precondition for the other, and it's not the kernel.

Here's why. A kernel produces a determination and an evidence artifact describing that determination. Chapter 2 already showed what that artifact is worth if it never leaves the kernel's own infrastructure: nothing, evidentially, because it's the operator's own account of the operator's own system. An EGK can be perfectly deterministic, perfectly tamper-resistant against the agent it governs, and fail safe every single time — and none of that touches the problem. The kernel's correctness and the kernel's evidentiary standing are two different things. The first is an engineering property. The second depends entirely on whether anything independent of the kernel ever touched the record.

So an Executable Governance Kernel deployed with no witness anywhere near it hasn't skipped a nice-to-have. It has shipped a governance mechanism that cannot prove, to anyone outside its own operator, that it ever did what it claims to have done. Ask that kernel's operator for proof after a dispute, and what you get back is the operator's own database — the same testimony-from-an-interested-party problem Chapter 1 opened with, now wearing a kernel's clothes instead of a homeowner's.

This means the witness is not something you add later, once the kernel is running and the budget allows for it. It has to exist first, or be standing there the moment the kernel goes live, because the first determination the kernel ever makes is exactly as unprovable as the ten-thousandth if nothing independent was watching from the start. There is no point in a kernel's operating life where its evidence "becomes" trustworthy retroactively. Either the witness was there from the first determination, or every determination made before the witness arrived has no evidentiary standing and never will.

Turn that around and it becomes a straightforward rule for anyone building toward autonomous execution: the witness is not a companion product to sell alongside a governance kernel. It is a precondition the kernel cannot function without, in any deployment where its output will ever be tested — by a court, an insurer, an auditor, or a counterparty who doesn't take the operator's word for anything. A kernel without a witness isn't a smaller version of a governed system. It's an ungoverned system that happens to produce very convincing paperwork.

This has a consequence worth stating directly, because it changes what "readiness for autonomous deployment" actually means. An organization moving toward agentic execution and asking "when do we need a witness layer" has the question backwards. The witness layer is not a step that comes after the kernel is built. It's infrastructure that has to already be standing, independent, and operating before the kernel's first determination is worth anything to anyone who wasn't in the room.

Which raises the obvious next question. If the witness has to exist before the kernel does, does it have to wait for the kernel at all? The next chapter answers that directly: no. The witness works on systems that have no kernel, no execution boundary, and nothing autonomous in them whatsoever — and it's already sellable, on its own, today.

Chapter 7 — The Factory Floor, No Kernel Required

Take a plant. Cold storage, or a chemical process line — somewhere temperature, pressure, or equipment state matters for safety and for what an insurer will pay out if something fails. Nothing autonomous runs here. No agent, no kernel, no execution boundary. Just sensors, a control system, and people running shifts.

Here's how it works today, almost everywhere. Sensors on the line measure temperature, pressure, valve position, cycle counts, whatever the process needs watching. All of that feeds into a historian database — software that logs the readings over time. The plant owns that historian. The plant administers it. The plant's own IT staff can open it, query it, and, if they ever needed to, edit it.

Nothing happens until something fails. A compressor overheats. A batch spoils. A valve sticks open and something gets damaged. Now there's a claim, and the insurer needs to know what happened.

What do they get? The plant's own historian logs. Detailed, timestamped, exactly the kind of record Chapter 3 already named: forensic exhaust. The insurer's forensic engineer reconstructs the failure from data the plant controls, cross-checked against maintenance records the plant also controls, and interviews with plant staff. This is the homeowner's smoke detector story again, just with more sensors and a bigger claim.

It doesn't have to be this way, and building the fix requires nothing this paper hasn't already specified.

Put a witness beside the historian. Not instead of it — beside it. The witness taps the same sensor feed, or the same historian output, and does exactly the five things Chapter 4 named: captures each reading or state change as it happens, chains it to the

one before, timestamps it against an external clock the plant doesn't control, stores it on infrastructure the plant's administrators can't reach, and exposes a way to check all of that without needing the plant's cooperation.

Nothing about the plant's operations changes. No governance. No permit or deny. No execution boundary anywhere. The witness doesn't touch the control system, doesn't slow anything down, doesn't gate a single valve. It only listens and records, exactly as Chapter 3 required.

What changes is what happens at claim time.

Instead of handing the insurer a historian database and asking them to trust it wasn't altered, the plant hands over a chained, externally-anchored record that provably existed, unmodified, at the moment of the incident. The insurer's forensic engineer isn't reconstructing from a database the plant could have edited. They're verifying a record neither party can dispute the integrity of, even if they still argue about what it means.

This is buildable today. It requires no autonomous system, no AI agent, nothing agentic anywhere in the building. It is a sidecar and an anchoring service, sold against a concrete, priceable cost: the six-week forensic reconstruction, the dueling expert reports, the claims dispute that drags because nobody can agree whose logs to trust. A witness beats a log at any speed, for the same reason a security camera beats a witness statement. This chapter is the proof that the requirement in this paper isn't waiting for agentic AI to matter. It already matters, on a factory floor, this quarter.

It is also worth being explicit about what building this actually involves, because "cryptographic witnessing infrastructure" sounds like a research programme and isn't one. Every one of the five primitives maps onto a component that already exists, off the shelf, in wide production use. Capture is a read-only tap on a sensor feed or a historian's existing export interface — the same integration a monitoring vendor performs on the first day of any deployment. The chain is a standard hash function applied to each record with the previous record's digest included; the libraries are mature, audited, and free. External time is a timestamp authority under RFC 3161,[9] or periodic publication of a root hash to a public ledger — both are commodity services with existing providers. Administrative separation is object storage held by a third party under credentials the witnessed party's administrators do not possess, which is an account configuration, not an invention. The disclosure interface is a published root and an open verifier that any outside party can run against it. Nothing on that list requires novel cryptography, custom silicon, or a research team. The work is integration — connecting five known components correctly and resisting the temptation to add a sixth — and it is measured in weeks, not years.

The next chapter takes the same architecture and moves it into a commercial relationship instead of a single building — because the same sidecar that proves what happened on a factory floor can also prove what a vendor promised, and did or didn't deliver.

[9] Adams, C., Cain, P., Pinkas, D., & Zuccherato, R. (2001). *Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)*, RFC 3161. Internet Engineering Task Force. Defines a protocol by which a Time Stamping Authority, independent of the requesting party, binds a hash to a time value and signs the result.

Chapter 8 — The Vendor Who Can Prove Their SLA

A cloud host promises 99.95% uptime. A logistics company promises on-time delivery. A call center promises average handling time under four minutes. All three sell a number. All three prove that number the same way: their own dashboard, their own monitoring tools, their own monthly report.

The client buying the SLA has no independent way to check any of it. They can audit — expensively, occasionally, usually only after something's already gone wrong enough to justify the cost. Mostly they just trust the vendor's report, because the alternative is running their own parallel monitoring of someone else's infrastructure, which is rarely worth the trouble.

This is the same gap as the factory floor and the same gap as the house fire. The party whose performance is in question is the only source of the record of that performance. Nobody built it this way on purpose. It's just what happens when nobody bothers to ask who's watching the watcher.

Fix it the same way. The vendor doesn't change what they do. They wire a witness into the systems already producing the numbers they sell — the uptime monitor, the delivery tracker, the ticket timestamps — and it does the same five things it did on the factory floor: captures each reading as it happens, chains it, timestamps it externally, stores it somewhere the vendor's own admins can't reach, and lets the client check all of it without asking the vendor's permission first.

Nothing about the vendor's operations changes. The uptime monitor still runs the same way. The witness just stands beside it, watching, unable to touch it, unable to be touched by it.

What changes is what the vendor can now say to a client. Not "trust our monthly report." Instead: here is an independently anchored record of every uptime check this month, and you can verify it yourself, without our cooperation, without our database, without taking our word for anything.

That's not a compliance cost. That's a sales pitch. A vendor who can say this is the only one in the room who can prove their number instead of reporting it. That's a premium, a shorter renewal negotiation, and a reason a risk-averse client picks them over a competitor selling the identical service on the identical infrastructure with the identical number, unproven.

None of this requires the client to change anything either. They don't install software, don't run their own monitoring, don't take anything on faith beyond checking a chain they can verify against a public anchor. The entire cost of the fix sits on the vendor's side, one time, and it pays for itself the first time a client raises an SLA dispute and the vendor produces a record neither party can argue with.

This is the same architecture as the factory floor, sold to a different buyer for a different reason. One product. Two markets. Neither one is waiting on autonomous anything.

The chapter after this one asks a harder question: what happens when enough vendors, enough factories, and enough of everything else start doing this at once, and their witnesses need to talk to each other.

Chapter 9 — When Witnesses Need to Talk to Each Other

One factory with a witness is a better factory. One vendor with a witness is a better vendor. Neither of those facts requires anything to change outside the building or the contract they sit in. That's the whole point of the last two chapters — the fix works alone, today, without waiting for anyone else to do anything.

But nothing stays alone for long once it works.

A client with one vendor wants proof from that vendor. A client with fifty vendors doesn't want fifty separate witnesses, each with its own format, its own dashboard, its own way of proving the same basic thing: this happened, unaltered, at this time. Fifty incompatible proofs are not fifty times more convincing than one. They're fifty times more work to check, which means in practice most of them won't get checked at all, and the whole exercise quietly turns back into trust.

The same problem shows up from the other direction. An insurer underwriting a supply chain — a dozen factories, three logistics providers, a handful of subcontractors — doesn't want a dozen bespoke evidence formats either. They want one thing they can point a tool at and get a straight answer: was this witnessed, is the chain intact, does the time check out. If every node in that chain built its own witness independently, with its own conventions, the insurer is back to doing by hand what the witness was supposed to do automatically.

This is not a hypothetical strain. It's the same strain that hit every isolated system that turned out to matter more once other people needed to talk to it. A company's internal network was fine on its own. It stopped being fine the moment that company needed to exchange mail with another company that had built its network differently. Nobody solved that by asking every company to adopt one company's network. They solved it with a protocol nobody owned, that every network could speak regardless of who built it.

Evidence is heading the same direction, for the same reason. A witness that only speaks its own format is a witness that works precisely once, for precisely one relationship. The moment a second party — a client, an insurer, a regulator, another vendor down the same supply chain — needs to check that same evidence for their own purposes, an unshared format is friction, and friction is exactly what a witness exists to remove.

This is worth being precise about, because it's easy to overstate. Nothing here requires every witness in the world to be the same witness, run by the same company, on the same infrastructure. That would just be centralizing the self-attestation problem this whole paper has been arguing against — one party, now bigger, still capable of editing the record. What's being described is narrower: a shared way of expressing what a witness is claiming — captured event, chain, external time, administrative separation,

disclosure — so that two witnesses built by two companies that have never spoken to each other can still be checked the same way, by the same tool, by a third party who trusts neither of them individually.

That's a mesh. Not one witness. Many witnesses, independently built, independently operated, agreeing on nothing except the shape of what they're claiming — which is exactly enough for a client, an insurer, or a court to check any of them without having to learn a new system every time.

What a mesh like that would actually require to work — and where the pressure to build one is likely to come from first — is the next chapter's question.

Chapter 10 — What the Mesh Requires, and Where It Starts

A mesh doesn't need every witness to be identical. It needs them to agree on what they're claiming, in a shape any outsider can check without learning a new system for each one. That's a narrower requirement than it sounds, and worth stating precisely, because vague versions of this idea collapse into either "everyone uses one vendor" or "everyone invents their own format" — and both of those are the mesh failing to happen.

What has to be shared is not the implementation. It's the claim. Every witness, regardless of who built it, is asserting the same five things from Chapter 4: this event was captured as it happened, it's chained to what came before it, it's timestamped against a clock nobody in the relationship controls, it sits on infrastructure the witnessed party can't reach, and it can be checked by a third party without the witnessed party's cooperation. A shared format doesn't need to standardize how a company chains its events internally. It needs to standardize how that chain is expressed well enough that a checker built once can verify a witness built by anyone.

This is a lower bar than it sounds, and a harder one than it looks. Lower, because it doesn't require consensus on cryptographic primitives, storage architecture, or vendor choice — only on the shape of the claim being made. Harder, because "only on the shape of the claim" is exactly the kind of thin, unglamorous standard that's easy to get almost right and easy to quietly violate without anyone noticing, the same way Chapter 5 showed real systems drifting off the five primitives one at a time.

Where does the pressure to actually build this come from first? Not from idealism about interoperability. From whoever has to check the most witnesses, most often, and eats the cost when they can't.

That's not always an insurer. It's whoever is managing the largest fragmented pool of counterparties, and that party changes depending on the relationship.

An insurer pricing risk across a supply chain is one example — a dozen factories, several logistics vendors, subcontractors two and three tiers removed, all producing evidence in incompatible formats. But a CFO managing five hundred suppliers has the identical problem before any insurer ever gets involved. A procurement team qualifying fifty vendors has the identical problem. A CTO wiring vendor performance data

into internal risk dashboards has the identical problem. None of these parties need to care about cryptography, protocols, or the abstract virtue of interoperability. They care about one thing: the cost of checking a counterparty's claims scales with the number of counterparties, and that cost is worse when every counterparty's proof looks different.

This has already happened once, in a nearby domain, and it's worth naming directly because it's the clearest precedent available. SOC2 and ISO 27001 did not become near-universal procurement requirements because auditors demanded it.[10] They became requirements because enterprise buyers got tired of evaluating security posture from fifty vendors fifty different ways, and converged on a shared attestation format that let a single checklist stand in for fifty bespoke reviews. Nobody centrally mandated SOC2. It won because it was cheaper for the buy side to demand one shape of proof than to keep evaluating fragmented ones. SBOMs are following a similar path in software supply chains — pushed initially by federal procurement rules, then adopted well beyond them for the same buy-side reason.

Evidence formats are heading the same way, and the underwriter is just one instance of a more general rule, not the rule itself.

The general rule: the first large-scale consumer of evidence — whichever party is drowning in the most fragmented pool of counterparty claims — forces standardization, because they have no other way to make verification affordable at their scale. Sometimes that consumer is an insurer pricing a supply chain. Sometimes it's a procurement department qualifying vendors. Sometimes it's an ERP vendor building evidence ingestion into a product used by thousands of enterprises at once, which multiplies the pressure rather than merely feeling it. The identity of the first mover is not the load-bearing part of this argument. The load-bearing part is that fragmentation has a cost, that cost scales with the number of counterparties rather than the value of what's being witnessed, and somebody with enough counterparties to feel that cost acutely will demand a shared shape of proof before they'll tolerate paying it forever.

This is the same pressure that turned isolated corporate networks into something that had to speak a common protocol, and it arrives from the same direction: not from the networks wanting to talk to each other, but from someone downstream — a customer, a regulator, an underwriter, a procurement team — needing to reach across all of them at once and refusing to do it fifty different ways.

Which means the mesh doesn't get built by the entities producing evidence. It gets demanded by the entities consuming it at scale — and there is more than one kind of entity in that category, arriving at the same demand independently, for reasons specific to their own function. That convergence, arriving from finance, procurement, compliance, and insurance simultaneously rather than from any single vertical, is a stronger reason to expect a mesh to form than underwriting pressure alone would have been.

[10] American Institute of Certified Public Accountants, *SOC 2® — SOC for Service Organizations: Trust Services Criteria* (AICPA, 2017; criteria revised 2022). ISO/IEC 27001:2022, *Information security, cybersecurity and privacy protection — Information security management systems — Requirements* (International Organization for Standardization). Neither is legally mandated for private-sector vendors in most jurisdictions; both became de facto conditions of enterprise procurement through buy-side demand rather than regulatory requirement.

What that means for liability specifically — not just fragmentation cost, but who is actually on the hook when something goes wrong and the evidence doesn't hold up — is the next chapter's question.

Chapter 11 — The Purchasing Agent That Was Denied

Take a concrete case. A company authorizes an autonomous purchasing agent to place vendor orders up to fifty thousand dollars without human sign-off. The agent proposes an order for three hundred forty thousand dollars. A vendor dispute follows. The company says the agent was never authorized to spend that much. The vendor's insurer gets pulled in to work out who eats the loss.

Without a witness, the only record of what happened is whatever the company's own systems say — and the company has every reason to shape that record favorably once money is on the line. This is Chapter 1's homeowner again, just with a purchasing agent instead of a smoke detector.

Here's what changes with a kernel and a witness both doing their jobs.

The instant the agent proposed the three-hundred-forty-thousand-dollar order, the kernel checked it against the agent's authority and returned a determination — before the order could commit, not after. That determination produced a record. The witness captured that record the moment it existed, chained it to the one before it, timestamped it against a clock the company doesn't control, and stored it somewhere the company's own admins can't reach.

What that record actually contains, stripped to the essentials — including the one detail that makes the chain in Chapter 4 real rather than asserted: each record carries a hash of the one before it, so altering any earlier record breaks every hash after it, visibly:

> proposed_action: purchase_order, vendor=X, amount=340,000.00 USD
>
> authority_scope: agent_id=PA-7, ceiling=50,000.00 USD
>
> standard_applied: spend_policy_v4.2
>
> determination: DENY
>
> reason: amount_exceeds_authority
>
> timestamp: external clock, to the millisecond
>
> prev_hash: a91f…228
>
> this_hash: 6d0e…551
>
> external_anchor: posted independently, shortly after

Two things about this record matter, and both trace straight back to the argument already built.

First: it says DENY. If the three-hundred-forty-thousand-dollar order actually shipped anyway, this record alone proves the order did not go through governance as claimed. That's not the company's word against the vendor's. That's an independently-held

record showing the kernel said no — which means somewhere between the denial and the shipment, something happened that has nothing to do with whether the kernel worked correctly. The agent was bypassed. The record was tampered with. A human overrode the kernel off the books. Each of those is now a specific, checkable failure, not a dispute about whose story to believe.

Second: the record was anchored externally, shortly after it was made, by the witness — not by the company. If the company had stored this record only on its own servers, none of the above would matter, hash chain or not — a chain only proves nothing was altered *after* it left the company's hands. If the company held the only copy, a court or an insurer would be right back to trusting the party under investigation, which is the exact problem the fire marshal exists to solve. The external anchor is what lets the vendor's insurer, or a court, confirm this exact record existed at this exact time without needing the company's cooperation and without taking the company's word for anything.

This is what Chapter 6 already argued, made concrete. The kernel proved a determination was made. The witness is what proves the record of that determination can't be quietly rewritten by the one party with a reason to want it rewritten.

What this means for how insurers actually price this kind of risk — not in the abstract, but in the specific numbers they ask for — is the next chapter.

Chapter 12 — What the Underwriter Actually Asks For

An underwriter pricing an autonomous system doesn't want a philosophy. They want numbers, and they want to know where the numbers came from.

Here's what that means in practice, once a kernel and a witness are both doing their jobs the way the last chapter described.

The witness's chain isn't just a liability record for one dispute. It's a running history of every determination the kernel has made, and a running history is exactly what an actuary needs to price risk instead of guessing at it. Five numbers fall directly out of that chain, without adding anything to the kernel or the witness that isn't already there.

How often the kernel says no. Every determination is DENY or PERMIT, timestamped and chained. Count them over a month, a quarter, a year, and you get a denial rate — how often the system proposes something outside its own authority. A denial rate of zero across every action type isn't automatically good news. It might mean the agent never goes near its limits. It might mean the kernel is misconfigured to wave everything through. Either way, an underwriter who can see the number can ask the right follow-up question. An underwriter who can't see it is pricing blind.

How often a human steps in and reverses the kernel. Every override — a human overturning a DENY, or overturning a PERMIT after the fact — is itself a determination, witnessed the same way. A high override rate on denials means the policy is too tight for how the business actually runs. A high override rate on permits means the kernel is letting things through that people, on reflection, don't want let through. Either pattern is a problem worth pricing differently. Neither pattern is visible without the record.

Whether the kernel was even running. A kernel that's offline governs nothing during the time it's offline, and an underwriter who doesn't know the uptime of the governance layer is underwriting a system that might have had no governance at all for stretches of time nobody can account for. The witness's chain shows this directly — a gap in the chain is a gap in governance, and it's visible as a gap, not hidden as a blank space in someone's story.

Whether the kernel is drifting toward its limits. Not every action that gets denied was close to the line, and not every action that gets permitted was far from it. How often proposed actions arrive near a constraint — even when they end up permitted — tells an underwriter whether the operation is running with margin or running hot. This is a leading indicator, not a lagging one, and it only exists because every proposal, not just the denied ones, left a record.

Whether compiled policy matches what the business actually intended. This is the slowest of the five to compute and the hardest to automate, because it means checking executed actions against the broader intent the policy was supposed to capture, not just the narrow rule the kernel enforced. It's the one number that requires a human to look at the record and ask "is this actually what we meant" — but it still depends entirely on the record existing to be looked at.

None of these five numbers require anything beyond what Chapters 4 through 11 already built. They are not a new instrumentation layer bolted onto the kernel for the underwriter's benefit. They are what a witnessed chain of determinations already contains, read differently by a different party for a different purpose.

This is the point worth landing directly. A governance architecture that needs extra instrumentation to produce its own compliance evidence is a governance architecture with an extra cost that someone, eventually, will try to cut. A governance architecture whose compliance evidence falls out of doing its job correctly has no such cost to cut. The five numbers above are not a feature. They're a byproduct of a kernel and a witness both working, and byproducts don't get optimized away, because nobody has to build them in the first place.

Here's what this changes for the underwriter, concretely. Instead of a security questionnaire asking whether a policy exists, or a certification confirming a system passed a test six months ago under conditions that may not hold today, the underwriter gets a running, checkable record of what the system actually did. That the record is checkable is itself a public fact rather than something they have to take on trust, and the five numbers above compute directly from it.

None of this is hypothetical positioning. It is already how the insurance market is behaving, in public, this year.

Insurers are not waiting for a mature actuarial model before acting — they are acting by narrowing what they'll cover. Commentary from insurance counsel describes carriers introducing what are being called absolute AI exclusions: coverage voided outright if AI use or development is involved anywhere in the loss chain, regardless of how the

rest of the policy is worded[11]. Standard-setting bodies for commercial general liability published new generative-A[12] Reporting on the London market describes insurers capping cyber payouts specifically tied to AI-related incidents, with sublimits set far below the total value of the underlying policy.[13] Two of these — the absolute exclusions and the cyber sublimits — are filed or drafted rather than universally in force; the ISO endorsements have been attaching at renewals since January. The distinction matters less than the direction. Carriers do not draft exclusion language for risks they believe they can price.

This is the insurance market doing exactly what this chapter predicted it would do to a self-reported system: not pricing the risk at all. An exclusion is not a price. It is a refusal to price, because there is nothing admissible to price against. That refusal is the commercial opening this paper has described throughout — a witnessed system gives an underwriter something to price where an unwitnessed one currently gives them a reason to exclude.

The same pressure is visible from the buyer's side, at the board level, not just the underwriting side. A 2026 enterprise survey found that roughly two-thirds of CIOs and CTOs consider themselves accountable for AI systems they do not fully control, and that the large majority of those same executives believe their organization's governance is not keeping pace with deployment.[14] Separate industry analysis has projected that a large share of enterprises may roll back or decommission autonomous AI agents within the next two years specifically because of governance failures, not because of model capability limits.[15] A central bank official has publicly raised the prospect of requiring circuit-breaker or kill-switch mechanisms for agentic AI operating in trading and pay-

[11]Lathrop GPM, "The AI Coverage Gap: What New Insurance Exclusions Mean for Your Business," May 4, 2026: some carriers, including WR Berkley, have drafted and filed "absolute" AI exclusions across D&O, E&O, and fiduciary liability policies, eliminating coverage for any claim based upon, arising out of, or attributable to AI use, deployment, or development — including claims of inadequate AI governance or oversight.

[12]Insurance Services Office / Verisk, *Generative Artificial Intelligence Exclusion Endorsements*, forms CG 40 47, CG 40 48, and CG 35 08, published July 2025 with a January 2026 edition date and attached at U.S. commercial general liability renewals from January 1, 2026. By April 2026 carriers including W.R. Berkley, Chubb, Travelers, Berkshire Hathaway, and Cincinnati Financial had filed to adopt these endorsements or proprietary AI exclusion language. ISO endorsement forms are filed for use across the commercial general liability market and adopted at each carrier's discretion.

[13]Financial Times, "Insurers move to cap cyber payouts related to AI and 'LLMjacking,' " April 22, 2026. Beazley and QBE proposed sublimits holding payouts on specified AI losses to roughly 10% of overall policy value; under QBE's draft wording, a cyber policy covering up to $5m would provide approximately $250,000 for losses arising from LLMjacking. Beazley confirmed the wording was still in development and had not been added to in-force policies at the time of reporting.

[14]IBM Institute for Business Value, global study of 2,000 C-level technology executives (January–April 2026): two-thirds of surveyed CIOs and CTOs report being held accountable for AI systems they do not fully control, and 77% report AI adoption is outpacing governance capability. IBM Newsroom, "New IBM Study Finds CIOs and CTOs Face Growing AI Control Gap as Enterprise Deployment Scales," June 8, 2026.

[15]Gartner, "Gartner Says Applying Uniform Governance Across AI Agents Will Lead to Enterprise AI Agent Failure," press release, May 26, 2026: predicts that by 2027, 40% of enterprises will demote or decommission autonomous AI agents due to governance gaps identified only after production incidents occur. Distinct from Gartner's June 25, 2025 prediction that over 40% of agentic AI projects will be canceled by end-2027; the 2026 release concerns governance-driven demotion of agents already in production.

ment systems, on the explicit grounds that human oversight cannot keep pace with agent-speed execution.[16]

None of these six data points — the exclusions, the CGL standard change, the sublimits, the executive survey, the rollback forecast, the regulator's kill-switch proposal — were written with this paper's argument in mind. They are six independent signals, from insurers, insurance counsel, standard-setting bodies, enterprise survey data, industry analysts, and a financial regulator, all pointing at the same gap: nobody currently has a way to prove governance was present at the moment of action, and every one of these parties is responding to that gap by pricing it out, excluding it, or threatening to regulate around it. That is the market discovering, from six different directions, the exact problem Chapter 1 opened with.

That changes coverage in three ways. Underwriters can make continuous witnessing a condition of coverage, the same way a fire insurer makes a working sprinkler system a condition of coverage — not as documentation of intent, but as a standing, checkable fact. They can price two systems differently even if they run identical software, because one produces a witnessed chain and one doesn't, and the difference between "we say it's governed" and "here's the record" is exactly the difference underwriting has always priced. And they can settle disputes against the record instead of against competing narratives, which is faster, cheaper, and harder to argue with than the alternative.

None of this is available to an underwriter pricing a self-reported system. All of it falls out of a witnessed one for free.

What happens once enough of this exists across enough organizations that the record itself becomes something people build businesses around — not just insurers pricing risk, but vendors, auditors, and eventually markets — is the subject of the next chapter.

Chapter 13 — The Market That Cannot Be Owned

Every chapter so far has built one witness at a time — a kernel, a factory, a vendor's SLA. Put enough of those in the world at once, and something bigger than any single deployment starts to take shape: a category, with a market underneath it.

Start with the size of it. Look back at every example this paper has used. A factory with an insurable risk. A vendor with an SLA. A kernel with a determination that needs proving. None of these are exotic. They are the ordinary condition of almost every commercial relationship that involves a promise, a risk, or a number one party reports to another. Every insurance policy on earth prices a promise it cannot independently verify. Every vendor contract with a performance clause prices a number the vendor self-reports. Every supply chain has a dozen parties whose word is the only record of

[16] Bank of England Deputy Governor Sarah Breeden, remarks at the European Central Bank's Sintra Forum, June 30, 2026: stated that existing regulatory frameworks were not built to contemplate autonomous agents, that reliance on human-in-the-loop oversight for all agent actions is unlikely to remain realistic, and floated market-wide circuit breakers or "kill switches" for agentic AI in trading and payments. Reported by Reuters and Computer Weekly, "Bank of England explores trading 'kill switches' to contain AI meltdowns," July 2026.

what they actually did. That is not a niche. That is the substrate underneath most of the economy — and right now, almost all of it runs on testimony, not evidence.

That is the addressable market. Not "companies building AI agents." Every insurable, auditable, or SLA-bound relationship that currently runs on self-reported records. The kernel and the autonomous agent are the newest reason this problem exists. They are not the biggest ones.

Now the harder question. Who gets to own this? Who holds the intellectual property? Who licenses this collection of functions? Nobody — no person, no company, no consortium, no government. And this is worth taking seriously, because it looks like it kills the business case.

A witness owned by a single commercial party is exactly the party with both the motive and the means to corrupt it. Say a company built the dominant witnessing platform and ran every anchor, every chain, every disclosure interface through infrastructure it alone controlled. That company is now the one party in the entire arrangement with something to gain from the record saying one thing rather than another — which is precisely the flaw this whole paper has been building a case against since Chapter 1. A witness that answers to a single owner has not solved the self-attestation problem. It has recreated it at a larger scale, with better marketing.

This is not a hypothetical risk to be managed with the right contract terms, a copyright, a license, a sales team. The moment ownership concentrates, independence collapses — for the same reason the kernel cannot prove itself. A protocol that can be owned is a protocol that will eventually be bent toward whoever owns it, whether or not that ever actually happens — because "could be bent" is enough to void the very property the market is paying for.

So the standard — the format, the hashing scheme, the shape of the claim from Chapter 10 — has to stay open. Nobody owns it. But anybody can build it, and they should.

That sounds like it removes the business opportunity. It doesn't. It relocates it.

Nobody owns TCP/IP,[17] and that didn't stop entire industries — content delivery, hosting, network security, monitoring — from building enormous, durable commercial value on top of a protocol nobody controls. Nobody owns the domain name system[18] in any commercial sense, and registrars, DNS security firms, and monitoring services built real businesses in the space around it anyway. Certificate Transparency logs are an open, auditable standard[19], and a genuine commercial ecosystem of monitoring and compliance tooling grew up around them without anyone owning the log format itself.

[17] Postel, J. (ed.) (1981). *Internet Protocol*, RFC 791; *Transmission Control Protocol*, RFC 793. Internet Engineering Task Force. Published as open standards with no patent or ownership claim; maintained by the IETF, a body with no membership requirements and no licensing authority over the protocols it stewards.

[18] Mockapetris, P. (1987). *Domain Names — Concepts and Facilities*, RFC 1034; *Domain Names — Implementation and Specification*, RFC 1035. Internet Engineering Task Force. The protocol specification is open and unowned; ICANN coordinates the root zone and accredits registrars under contract but holds no proprietary right in the protocol itself, which any party may implement without permission or fee.

[19] Laurie, B., Langley, A., & Kasper, E. (2013). *Certificate Transparency*, RFC 6962; superseded by Laurie, B., Messeri, E., & Stradling, R. (2021). *Certificate Transparency Version 2.0*, RFC 9162. Internet Engineering Task Force. An append-only, publicly auditable Merkle-tree log format with no owner; logs are operated

This is not only precedent from other domains. It has already started happening inside this one. At least one decision-record standard in this space — the same one Chapter 5 noted — has been placed in the public domain outright, held by a nonprofit foundation whose charter exists to prevent any party, including its own founder, from ever privatizing it. Whatever else that standard does or doesn't satisfy, its licensing structure is the ownership argument of this chapter, reached independently, before this chapter was written.[20] The split this paper predicts is not a proposal waiting for a first mover. The first mover already moved.

The same split applies here. The protocol stays unowned. The value sits one layer up, in four places specifically:

Deployment. Somebody has to actually install and operate the sidecar across a client's infrastructure — the factory's sensors, the vendor's uptime monitor, the kernel's determinations. That is a service, and it is billable.

Dashboards. A hash chain and a Merkle proof mean nothing to a risk manager or a CFO. Somebody has to make the proof legible — a dashboard that says, in plain terms, this is witnessed, this is intact, here is what changed — without requiring the reader to understand cryptography.

Integration. Wiring a witness into an existing SLA, an existing insurance workflow, an existing compliance process is not automatic. It is systems work, done once per client, and it is exactly the kind of service work that turns into long contracts.

Node operation. Somebody has to actually run the anchoring infrastructure, reliably, at scale, across however many clients are depending on it. That is real infrastructure, with real uptime requirements, and it is a real business.

None of those four require owning the standard. All four require building something on top of it that a client will pay for, on an ongoing basis, because the alternative is going back to self-reported testimony.

This is the pitch worth stating plainly to anyone building in this space. Do not try to own the evidence standard. It cannot be owned without breaking the thing that makes it valuable, and any attempt to own it is a bet against the paper's own central argument. Own the tooling. Own the dashboards. Own the service relationships that make an open, unowned standard usable by people who have no interest in reading a Merkle proof themselves. That is where the durable business is, and it is a bigger business than owning the protocol would have been, because it is the same business every open protocol before this one has already proven out.

What happens once this actually starts running at scale — once enough witnesses, in enough industries, are speaking the same shared claim — is the mesh Chapter 9 already

independently by multiple parties, and monitoring and compliance tooling built on the format is commercial while the format itself is not.

[20] Genesis Glass Foundation / Fondation Genèse Cristal, *Corporate Charter and Articles of Incorporation*, accountability.ai/charter (2026). AgDR protocol held in perpetual trust under CC0 / Apache 2.0 dual license; Foundation incorporated as a Canada Not-for-profit Corporation, Corp. No. 1779509-2, with stated purpose of preventing privatization or restriction of the standard.

named. What that mesh does to the rest of the economy, once it is load-bearing rather than incidental, is where this paper's argument ends and the next one picks up.

Chapter 14 — Evidence as a First-Class Enterprise Data Type

Every enterprise software system in existence is organized around a small set of things worth tracking permanently. Money is one. Inventory is one. Customers are one. Employees are one. These are not arbitrary categories. They earned their place because a business cannot run without a system of record for each of them, and once a category becomes a system of record, an entire industry grows up around storing it, moving it, and reporting on it.

Evidence is not yet one of those categories. This chapter argues it has to become one, and that the shape of that argument is not new — it is the same shape every other category went through on its way to becoming infrastructure.

Start with what a system of record actually does, stripped of the vocabulary vendors use to sell it. A system of record is the place an organization keeps the version of something it will stand behind if challenged. Financial records exist because a company has to be able to answer, with authority, what it earned and what it spent. Inventory records exist because a company has to be able to answer what it has and where it is. Customer records exist because a company has to be able to answer who it dealt with and on what terms. None of these categories were designed in advance and then adopted. They became categories because the cost of not having an authoritative answer eventually exceeded the cost of building one.

Evidence — in the sense this paper has used the word since Chapter 1 — is heading down the identical path, for the identical reason. An organization running autonomous systems, or dealing with vendors who run them, is going to be asked, sooner or later, a question it currently has no authoritative way to answer: not "what happened," but "prove it happened, independently of your own say-so." Right now that question gets answered the way Chapter 3 already described — a log, pulled from wherever it happens to live, produced by the party being asked to answer for it. That is not a system of record. That is testimony with a timestamp.

The chapters before this one built the mechanism that fixes this — a witness, disjoint from what it observes, producing a chained and externally anchored record. What this chapter argues is narrower and more practical: once that mechanism exists at any scale, it does not stay a bolt-on sidecar. It becomes something an enterprise tracks the same way it tracks money and inventory, because the same forcing function applies. The cost of not having an authoritative, independently checkable answer to "prove it" eventually exceeds the cost of building the system that provides one.

This has a direct and unglamorous consequence for where evidence actually lives. Financial data lives in a general ledger, not a folder of receipts. Inventory data lives in a warehouse management system, not a whiteboard. Customer data lives in a CRM, not a rolodex. Each of these transitions happened because the category became important enough that ad hoc storage stopped being tolerable. Evidence is currently stored the

way receipts were stored before the general ledger existed — scattered, system-specific, owned by whichever team happened to generate it. That does not survive contact with scale any better than paper receipts did.

The practical question this raises is where evidence actually gets stored, queried, and reported on inside a real organization — and the honest answer is that it will end up wherever the other systems of record already live: the enterprise resource planning layer. Not because ERP vendors are especially well positioned to build witnessing infrastructure themselves — Chapter 13 already established why the protocol has to stay unowned, and an ERP vendor owning the witnessing standard would be exactly the concentration of ownership that chapter warned against. The claim is narrower: whatever standard the mesh converges on, the place enterprises will want to see it surfaced, queried, and reported against is the same system already holding their financial and operational records, because that is where every other system-of-record category has ended up living.

This produces a specific and testable prediction about how adoption actually spreads, and it is worth naming because it changes who the paper's argument is actually aimed at. The mesh does not spread primarily because individual vendors decide, one at a time, to build a witness into their SLA monitoring. It spreads because procurement teams start asking for it as a condition of doing business, the same way they now ask for a SOC2 report or a software bill of materials before signing a vendor contract. A procurement question that currently reads "send us your compliance documentation" becomes, over time, "expose your evidence feed" — a machine-checkable claim instead of a PDF someone has to trust.

This is not a hypothetical progression. It is close to the path SOC2 and SBOMs already took. SOC2 was never centrally mandated at all; it became near-universal because enough buyers, independently, got tired of evaluating vendors on trust and converged on a shared format that let one checklist stand in for fifty bespoke reviews. SBOMs got their initial push from federal procurement requirements, then spread the same way — buyers well outside the mandate adopted the format because checking one shape of proof was cheaper than checking fifty. Evidence feeds are the next entry on that list, and the mechanism forcing their adoption is the same: a buyer with enough vendors to feel the cost of fragmentation, demanding a shared shape of proof because the alternative no longer scales.

Once that demand exists at procurement scale, the distribution channel writes itself. Enterprise platforms — the SAPs, the Oracles, the Workdays, the Salesforces, the ServiceNows of the world — do not need to invent *evidence witnessing*. They need to do what they have always done with every new category that became load-bearing: build the ingestion, storage, and reporting layer for it, and sell that layer as a feature of the platform enterprises already run everything else through. This is not speculation about a distant future. It is the observation that every prior data category which became infrastructure got absorbed into the enterprise stack the same way, and there is no structural reason evidence would be the exception.

None of this changes anything established in the chapters before it. The protocol still has to stay unowned, for exactly the reasons Chapter 13 gave. What changes is where the paper's commercial argument actually lands. The opportunity is not merely "build a witnessing sidecar and sell it to individual vendors one at a time." It is building the ingestion and reporting layer that lets evidence become a queryable, standing category inside the systems enterprises already depend on — the same commercial position financial reporting software occupied before anyone thought to call it indispensable, and the same position inventory and customer data occupied after that.

The next chapters test this argument, and the paper's other claims, against the strongest objections available.

Interlude — The Regulator Was Never Asking for a Kernel

Every major jurisdiction building an AI regulatory framework has independently arrived at the same short list of words: traceability, auditability, accountability, oversight. The EU AI Act asks for it. The NIST framework asks for it.[21] The UK, Singapore, Canada, and a half-dozen others building national AI governance regimes ask for versions of the same thing, in their own language, without having coordinated with each other.

None of them specify how. That's not a gap in drafting. Regulators writing instruments meant to last a decade have good reason not to mandate a specific architecture — mandate the wrong one, and the law is obsolete before it's tested in court. So the requirement stays at the level of outcome: prove the system was accountable, prove its behavior can be traced, prove someone was watching. What satisfies "prove" is left for whoever gets audited to work out.

This is where most organizations get the answer wrong, not out of bad faith but out of habit. The instinct is to satisfy "traceable" and "auditable" with better logging — more detail, longer retention, a cleaner dashboard for the auditor to look at. Chapter 3 already named what that actually is. A more detailed log is still the operator's own account of the operator's own system, and an auditor reading it is doing exactly what an insurer reading a self-owned historian is doing: trusting the party under review to have told the truth about itself.

A witness satisfies the same words differently, and it satisfies them better by construction rather than by effort. "Traceable" doesn't require more logging — it requires a chain, external time, and a disclosure interface an auditor can check without the operator's cooperation, all of which Chapter 4 already specified as the minimum for anything to count as evidence at all. "Auditable" doesn't require a bigger compliance team — it requires exactly the running record Chapter 12 already showed an underwriter reading

[21] Regulation (EU) 2024/1689 (the AI Act), Art. 12 (record-keeping: automatic logging of events over the lifetime of a high-risk system) and Art. 13 (transparency and provision of information to deployers); see also Art. 19 on retention of automatically generated logs. National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1 (January 2023), MEASURE and MANAGE functions, which call for documented, traceable measurement of AI system performance without specifying an implementing architecture. Neither instrument prescribes a mechanism by which the required record is to be produced or verified.

for denial rate and override rate, except now the reader is a regulator asking a different question of the identical record. "Accountable" doesn't require a new attestation process — it requires the record to exist independently of the party being held accountable, which is the entire and only point of Chapter 6's argument that the witness has to be there before the kernel's first determination, not added afterward to make the paperwork look complete.

This is worth stating plainly, because it means something specific and useful to a compliance officer reading this paper rather than a philosopher: a properly built witness is not an AI-specific compliance investment. It is the same infrastructure Chapter 7 sold to a factory and Chapter 8 sold to a vendor, pointed at a regulator instead of an insurer or a client. The five primitives don't change. The chain doesn't change. What changes is only who is on the other end of the disclosure interface asking to check it.

None of this requires reading every regulation in every jurisdiction and mapping five primitives onto each one's specific clause numbers — that is real work, and it belongs to whoever is deploying into a specific regulatory regime, not to this paper. What this paper claims is narrower and holds regardless of jurisdiction: every regulator asking for traceability and auditability is asking, in different words, for the same thing an insurer asks for and the same thing a fire marshal has always asked for — a record that doesn't depend on the word of the party being investigated. Build the witness once, correctly, and the regulator's question was already answered before it was asked.

Chapter 15 — Objections

Every argument built this far has moved in one direction: toward the claim that a witness must be structurally disjoint from what it observes, or it isn't a witness at all. Before this paper closes, that claim has to survive its own strongest counterarguments — not the weak version, the version a skeptical builder would actually raise.

Objection one: This is audit logging with a cryptography budget.

Strip away the vocabulary — chain, external anchor, disclosure interface — and isn't this just a log with a hash function bolted on? Companies have been hashing log files for integrity checks for decades. Calling it a "witness" doesn't make it a new category.

The objection is right about the mechanics and wrong about what they add up to. A hash on a log file proves the file wasn't altered after the hash was taken. It says nothing about who took the hash, where it was stored, or whether the party who could edit the underlying log could also just regenerate the hash to match. A log with a hash, stored on the same infrastructure as the system it describes, administered by the same people — has changed nothing about who's making the claim. It has made the claim look more rigorous without making it more independent. Chapter 4's five primitives aren't cryptography for its own sake. Chain and hashing are one primitive out of five. The other four — external time, administrative separation, disclosure without cooperation, capture without request — are the ones doing the actual work, and none of them are satisfied by hashing a file. A system that has all the cryptography and none of the separation has decorated a log. It has not built a witness.

Objection two: An unowned standard just relocates the trust problem to whoever runs the anchoring infrastructure.

Chapter 13 argued the protocol has to stay unowned, but somebody still runs the servers that hold the anchors, somebody still operates the node that posts the chain externally. Isn't that party now the single point of trust the whole architecture was built to avoid?

This is the sharpest version of the objection, and it deserves a straight answer rather than a dismissal. The node operator is not asked to be trusted the way the witnessed party was asked to be trusted. The difference is disclosure and redundancy. A witnessed party's record is unverifiable by construction — nobody outside the witnessed party can check it without that party's cooperation. A node operator's anchor is, by the same five primitives, independently checkable by anyone, and nothing stops multiple independent node operators from anchoring the same chain to different public ledgers simultaneously. A corrupt node operator who alters what they're anchoring produces a visible mismatch the moment a second, independent anchor is checked against it. A witnessed party who alters their own record produces no such mismatch, because there was nothing independent to check it against in the first place. The node operator's trust requirement is falsifiable and distributed. The witnessed party's was neither.

Objection three: The mesh hypothesis is unfalsifiable as stated.

Chapters 9 and 10 argue evidence systems are heading toward a shared, interoperable standard the way isolated networks converged on shared protocols. But this is a prediction about the future, dressed as an observation about the present. What would actually prove this wrong?

A fair challenge, and the honest answer is that the mesh hypothesis would be falsified if fragmentation persisted indefinitely without cost — if insurers, courts, procurement teams, and clients kept tolerating bespoke, incompatible evidence formats without any of them ever demanding a shared one, no matter how large the fragmented pool of risk grew. Chapter 10's argument was never that a mesh is inevitable by nature. It was that whichever party is drowning in the largest fragmented pool of counterparty claims — an insurer, a CFO, a procurement team, an ERP vendor — has a cost that scales with fragmentation and would rather not pay it. If none of those parties ever materialize, or materialize and tolerate the cost indefinitely, the mesh doesn't happen. That is a testable claim, not an article of faith. It just hasn't been tested yet, because the volume of independently-built witnesses in the world today isn't yet large enough for the fragmentation cost to bite.

Objection four: A witness whose output triggers a financial consequence has stopped being disjoint.

Some systems already built in this space couple economic consequences directly to what a witness reports — capital locked against a transaction, forfeited or released depending on what the witness later confirms happened. Doesn't a witness whose report now has a party financially interested in its content stop being independent, whatever Chapter 4 says about its five primitives?

No, and the reason is a distinction the objection collapses: measuring a value is not the same act as moving it. A witness's output is an evidence receipt — a cryptographically verifiable claim that a specific event occurred, unaltered, at a specific time. A receipt is not a payment. It does not transfer funds, release capital, or execute anything. It is closer to a shipping manifest or a utility meter reading: a factual record that another system may later use when determining what action, if any, should occur.

The party who acts on it is not the witness. It is a second, distinct mechanism — an execution layer that takes the receipt as an input, evaluates it against a standard (did the qualifying event occur, as attested), and produces its own binding output: release, or forfeit. That mechanism is a kernel in every sense Chapter 2 already defined. It has a proposed action, a standard, a determination. And because it is a kernel, it needs its own witness, for exactly the reasons Chapter 6 already gave — its determination is worthless as evidence if nobody independent of it ever touched the record.

State the separation directly, because it is the load-bearing part. The witness produces evidence. The execution layer determines admissibility. The settlement layer transfers value. The enforcement layer resolves failures. These are four distinct architectural responsibilities, and collapsing any two of them compromises the independence of all four.

The same separation applies whether the downstream consequence is payment, insurance, regulatory action, access control, a contractual penalty, or legal liability. Nothing in this argument is specific to money. Money is only the case where the coupling is most visible, because the consequence arrives fastest and someone can name its exact size.

So what looked like a single corrupted category is actually four responsibilities — three of them already named in this paper — sitting in sequence rather than in one box. The witness measures and stays disjoint, exactly as Chapter 4 requires — it would produce the identical receipt whether or not anything was riding on it, because it has no visibility into what happens downstream and no capacity to act on it even if it did.

The witness remains a measuring instrument. Economic systems consume its measurements; they do not redefine its function. Independence is preserved because evidence generation, determination, settlement, and enforcement remain separate architectural concerns.

What remains, once these objections are weighed rather than dismissed, is the subject of the closing chapter.

Chapter 16 — Prior Art and Honest Placement

This paper does not claim to have invented the idea of an independent witness.

That claim would be false, and a careful reader would find it so quickly. The concept of a record kept by someone other than the party being recorded is older than any of the technology in this paper. It appears under different names in different disciplines, for different reasons, long before anyone connected them into a single category. Honest

engagement with that history is not a weakness here. It is a precondition for the rest of this paper being taken seriously.

The oldest and plainest precedent is the one this paper opened with. Fire insurance has required independent origin-and-cause investigation for centuries, resting on the same unstated rule this paper has spent its opening chapters making explicit: the party whose behavior is in question cannot be the sole source of the record of that behavior. Sprinkler system inspection, pressure vessel certification, maritime safety equipment audits — every regulated physical-risk domain has some version of the same requirement, enforced by a party with no stake in the outcome. None of these were built with cryptography, chains, or external anchors. They didn't need to be. The requirement predates the tools that later made it possible to satisfy at machine speed.

The more recent and more directly relevant precedent is the set of cryptographic witnessing systems referenced in Chapter 5. Guardtime's hash-aggregation architecture predates this paper by well over a decade, built for an entirely different purpose — Estonian government record integrity — with no interest in autonomous systems or agentic execution. Independently built runtime-governance and witnessing systems have arrived at comparable architectures in close succession, across domains, without contact between the teams responsible. An open, public-domain decision-record standard emerged in the same window, stewarded by a nonprofit foundation and built for evidentiary admissibility under Canadian law — arriving at chained, signed, inference-time capture without deriving the five primitives, and anticipating this paper's ownership argument in its licensing structure before this paper made it. These systems did not derive the five primitives in Chapter 4 from first principles the way this paper has. They arrived at something close to them by solving a narrower, domain-specific version of the same problem. This paper's contribution is not the primitives themselves in isolation — it is the derivation of why exactly five, no more and no fewer, are necessary regardless of domain, and the argument that a kernel of the kind established in prior work cannot function without exactly this structure standing beside it.

What this paper adds to that existing body of work is fivefold, and each piece is worth naming plainly rather than left implicit.

First, the disjointness requirement, stated as an absolute rather than a design preference. Existing witnessing systems vary in how strictly they hold this line — some tolerate a signature step on the hot path, some tolerate shared administrative infrastructure with the system they observe, treating these as minor implementation details rather than disqualifying failures. This paper treats them as disqualifying, because Chapter 3 already showed why: the instant a witness gains any power to act back on what it observes, it has stopped being a witness. That is not a stricter version of an existing standard. It is a different standard, and it is the one this paper argues the category actually requires.

Second, the sequencing argument in Chapter 6 — that the witness is not a companion to a governance kernel but a precondition for one. This is not how existing witnessing systems are typically marketed or deployed. They are usually sold as an added layer of assurance on top of a system that is already presumed to work. This paper's claim is

stronger and less comfortable: a kernel's evidentiary standing is zero without the witness, from its first determination onward, and there is no point at which that standing can be retroactively repaired.

Third, the ownership argument in Chapter 13. Most existing witnessing systems are commercial products, owned outright by the companies that built them. This paper argues that ownership is structurally incompatible with the independence the category requires, and that the durable commercial opportunity sits one layer above the standard rather than in owning it. This is a direct challenge to how several existing systems are currently positioned, and it is stated here as a challenge rather than a criticism: any system built as a proprietary, owned witnessing platform has built something valuable, but not something that can do the one job a witness has to do without eventually being asked to compromise it.

Fourth, the infrastructure argument in Chapter 14. Existing witnessing systems are, without exception, sold and deployed as bolt-on sidecars — an add-on layer sitting beside a system that already works. This paper argues that the category is heading somewhere more structural: toward a first-class system of record inside the enterprise stack, distributed through the same platforms that already hold financial, inventory, and customer data, and adopted through the same procurement pressure that made SOC2 and SBOMs near-universal. No existing witnessing system is currently positioned this way, and that positioning — not just the technology — is where a meaningful part of the durable commercial opportunity actually sits.

Fifth, transparent failure as a constitutional property rather than an implementation choice. Existing witnessing systems treat the question of what a witness does when it cannot attest as an operational detail — a monitoring alert, a retry, a gap someone notices later. This paper derives it from the witness's purpose: a mechanism whose only function is to let an outsider distinguish a genuine record from a fabricated one cannot, under any failure condition, report confidence it does not have. The requirement follows from the job, not from the deployment. No existing system in this space states it as a requirement, and several fail it silently by design.

The honest placement of this paper, then, is not as the origin of the witness concept. It is a synthesis: taking a requirement that has existed in fragmented form for centuries, deriving its minimal structure from first principles rather than reverse-engineering it from existing products, and following that derivation to conclusions — on sequencing, on ownership, on infrastructure, on what counts as disqualifying — that existing implementations have not consistently reached on their own.

That is the paper's claim to originality. It is a narrower claim than inventing the category. It is also a defensible one.

End of manuscript — Version 2.0

Evidence Systems: *Why Independent Witnesses Emerge at Machine Speed*

16 chapters. 1 interlude.

Jonathan M. Watson
Independent Researcher
jonmwatson@gmail.com
August 01, 2026

Glossary of Terms

Terms are defined as used in this paper. Where a term is also used elsewhere with a different meaning, that usage is not implied here.

Administrative separation The requirement that a witness's infrastructure — its credentials, storage, and keys — be unreachable by whoever administers the system being witnessed. Not merely harder to reach or reachable-but-logged. *Unreachable*. One of the five primitives of the minimal witness. Its absence is the most common way a witnessing system quietly fails without anyone noticing.

Capture The primitive by which a witness takes in an event — a determination, a sensor reading, a state change — the moment it occurs, without requesting it and without the witnessed party controlling when or whether it happens. Capture on a schedule set by the witnessed party is not capture; it is being told a story.

Chain The primitive linking each captured record to the one before it, such that altering any single record breaks the chain visibly at that point. An unchained record is indistinguishable from a diary entry: writable at any time, with no way to tell when it was actually written.

Coupling The condition in which a witness's output triggers a downstream financial or operational consequence — capital released or forfeited, a contract term satisfied, a penalty applied. Resolved in this paper by distinguishing measurement from movement, and by separating four responsibilities that a coupled system tends to collapse into one: the witness produces evidence, the execution layer determines admissibility, the settlement layer transfers value, and the enforcement layer resolves failures. The witness measures and remains disjoint; the mechanisms downstream of it are each accountable to their own governance and their own witness.

Disclosure interface The primitive that allows a third party — an auditor, a court, an insurer, a counterparty — to verify a witness's chain, timestamps, and content without needing the cooperation of the witnessed party. A disclosure mechanism that requires the witnessed party to unlock anything has made the witness dependent on the party it exists to check.

Disjointness The structural requirement that a witness only listen and record — that it cannot gate, delay, permit, deny, advise, or otherwise influence the system it observes, and has no write path back into it. The dividing line this paper treats as absolute rather

than a matter of degree: the instant a witness gains any capacity to act on what it observes, it has stopped being a witness and become an unacknowledged second execution boundary.

Enforcement layer The mechanism that resolves failures once a determination has been reached and any settlement has occurred — penalty, revocation, escalation, or referral to an external authority. Distinct from the settlement layer, which transfers value, and from the execution layer, which determines admissibility. Not developed in this paper; named to complete the four-way separation the coupling objection requires.

Evidence In this paper, a record that satisfies all five primitives of the minimal witness — captured independently, chained, externally timestamped, administratively separated from the party it describes, and disclosable without that party's cooperation. Distinguished sharply from testimony and from a log, both of which may be detailed and sincere but remain the account of an interested party.

Evidentiary standing The property, possessed or not possessed by a record, of being usable as proof to a party that was not present when the record was made. A kernel's determination has no evidentiary standing until an independent witness has touched it; standing cannot be established retroactively for determinations made before the witness existed.

External time reference The primitive requiring that a witness's timestamp come from a clock the witnessed party cannot adjust, backdate, or pause. A timestamp generated by the witnessed system's own clock is testimony with a number attached, not evidence.

Forensic exhaust A description of what a conventional log actually is: a byproduct of a system having run, generated for the system's own convenience, in its own custody, read by its own tools. Operationally useful; evidentially worthless, because it was never independent of the party it describes.

Interested party Any party whose own conduct is the subject of a record it also produced. The structural reason testimony from such a party cannot count as evidence, regardless of whether the party is being honest — the record cannot distinguish an honest account from a shaped one.

Kernel A deterministic mechanism, established in prior work, that evaluates a proposed action against a compiled standard and returns a binding permit-or-deny determination before the action commits. This paper does not re-derive the kernel's properties; it assumes them and addresses a problem the kernel cannot solve for itself — proving, to a party that was not present, what it decided.

Log A system's own account of its own operation, kept on infrastructure it owns, for its own convenience. Distinguished from a witness by the fact that a log can be produced by the system it describes, while a witness — by construction — cannot.

Mesh The condition in which independently built witnesses, agreeing on nothing but the shape of what they claim — captured event, chain, external time, administrative separation, disclosure — can be checked by the same tool without requiring the checking party to learn each witness individually. Not a single witness scaled up; many witnesses, interoperable by shared claim-format rather than shared ownership.

Minimal witness The five-primitive mechanism this paper derives as the smallest configuration capable of making a record count as evidence: captured event, chain, external time reference, administrative separation, disclosure interface. Deliberately incapable of investigating, judging, or establishing the truth of what it records — only that a specific record existed, unaltered, at a specific time.

Node operator The party that runs the infrastructure anchoring a witness's chain externally. Distinguished from the witnessed party in that a node operator's output is independently checkable by anyone, and multiple node operators can anchor the same chain redundantly — making the node operator's trust requirement falsifiable and distributed, unlike the witnessed party's.

Receipt (**evidence receipt**) The output of a witness: a cryptographically verifiable claim that a specific event occurred, unaltered, at a specific time. It transfers, releases, and executes nothing. Closer to a shipping manifest or a utility meter reading than to a payment — a factual record a separate execution layer may later act on. Distinguishing the receipt from settlement is what resolves the apparent tension between witness independence and economic consequence.

Self-attestation problem The condition in which the only account of a system's behavior comes from the system's own operator. First introduced with the fire-insurance homeowner in Chapter 1; shown in Chapter 2 to persist even when the system being attested to — a kernel — is itself flawless, because the kernel's correctness and its evidentiary standing are separate properties.

Settlement kernel A second, distinct mechanism, downstream of the witness, that consumes a receipt as an input, evaluates it against a standard, and produces its own binding determination — release or forfeit. It performs the execution-layer function in the four-way separation; the transfer of value itself happens in the settlement layer beneath it. Governed by the same requirements as any kernel: it needs its own witness, for the same reasons the original kernel does.

System of record The place an organization keeps the version of something it will stand behind if challenged. This paper argues evidence is becoming one of these categories, alongside financial, inventory, and customer data, rather than remaining a bolt-on sidecar product.

Testimony An account produced by an interested party. May be entirely true and still structurally unusable as proof, because nothing in the account itself distinguishes an honest version from a shaped one.

Transparent failure The requirement that a witness, when it cannot establish the integrity of its own evidence, disclose that it cannot attest rather than report confidence it does not have. The disclosure is itself captured, chained, and anchored. Distinguished from failing closed, which is the correct behavior of an execution boundary, and from failing open in the conventional fault-tolerance sense, which this paper does not use.

Witness A mechanism that observes a system's behavior without being that system, without sharing its infrastructure, and without any ability to reach back into what it

observes. Satisfies the five primitives of the minimal witness; the paper's central subject from Chapter 3 onward.

End of Glossary

Bibliography — Version 2.0
Evidence Systems: Why Independent Witnesses Emerge at Machine Speed

A — Fire Investigation and Forensic Standards

National Fire Protection Association (2024). NFPA 921: Guide for Fire and Explosion Investigations. NFPA.

Daubert v. Merrell Dow Pharmaceuticals, Inc., 509 U.S. 579 (1993). U.S. Supreme Court.

Werth v. Hill-Rom, Inc., 856 F. Supp. 2d 1051 (D. Minn. 2012).

B — Cryptographic Witnessing and Timestamping

Guardtime (2016). Estonian Government, Guardtime Accelerate Adoption of Blockchain Technology to Secure 1M Patient Health Records. Press release, March 3, 2016.

e-Estonia. The Use of the National Blockchain Infrastructure to Support the E-Residency Initiative in Estonia. Interoperable Europe Portal, European Commission.

SDxCentral (2018). Verizon to Use KSI Blockchain Technology Developed for Estonia.

Genesis Glass Foundation / Fondation Genèse Cristal (2026). Agentic Genesis Decision Record (AgDR): Open Standard for Court-Admissible AI Decision Records. accountability.ai

Genesis Glass Foundation / Fondation Genèse Cristal (2026). Corporate Charter and Articles of Incorporation. Canada Not-for-profit Corporations Act, Corp. No. 1779509-2. accountability.ai/charter

C — Runtime Governance and Agent Authorization

ARMO (2026). AI Agent Governance: From Policy Framework to Runtime Enforcement.

TrigGuard (2026). Runtime Authorization for AI Agents. trigguardai.com/runtime-authorization-ai-agents

Business Wire (2026). Arcade Raises $60M to Become the Secure Action Layer Behind Every Production AI Agent. June 15, 2026.

D — Open Protocols and Standards

Postel, J. (ed.) (1981). Internet Protocol. RFC 791. Internet Engineering Task Force.

Postel, J. (ed.) (1981). Transmission Control Protocol. RFC 793. Internet Engineering Task Force.

Mockapetris, P. (1987). Domain Names — Concepts and Facilities. RFC 1034. Internet Engineering Task Force.

Mockapetris, P. (1987). Domain Names — Implementation and Specification. RFC 1035. Internet Engineering Task Force.

Laurie, B., Langley, A., & Kasper, E. (2013). Certificate Transparency. RFC 6962. Internet Engineering Task Force.

Laurie, B., Messeri, E., & Stradling, R. (2021). Certificate Transparency Version 2.0. RFC 9162. Internet Engineering Task Force.

Adams, C., Cain, P., Pinkas, D., & Zuccherato, R. (2001). Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). RFC 3161. Internet Engineering Task Force.

E — Insurance and Underwriting

Lathrop GPM (2026). The AI Coverage Gap: What New Insurance Exclusions Mean for Your Business. May 4, 2026.

Insurance Services Office / Verisk (2026). Generative Artificial Intelligence Exclusion Endorsements. Effective January 1, 2026. Forms CG 40 47, CG 40 48, CG 35 08.

Financial Times (2026). Insurers Move to Cap Cyber Payouts Related to AI and 'LLM-jacking.' April 22, 2026.

F — Enterprise Governance and Regulatory Commentary

IBM Institute for Business Value (2026). CIO and CTO Study: The AI Control Gap. IBM Newsroom, June 8, 2026.

Gartner (2026). Applying Uniform Governance Across AI Agents Will Lead to Enterprise AI Agent Failure. Press release, May 26, 2026.

Bank of England (2026). Remarks by Deputy Governor Sarah Breeden, ECB Sintra Forum, June 30, 2026. Reported in Reuters and Computer Weekly, Bank of England Explores Trading 'Kill Switches' to Contain AI Meltdowns, July 2026.

G — Independent Runtime Governance Architectures (2026)

Errico, H. (2026). *Autonomous Action Runtime Management (AARM): A System Specification for Securing AI-Driven Actions at Runtime*. arXiv:2602.09433 [cs.CR], February 10, 2026.

Mazzocchetti, A.M. (2026). *Cryptographic Runtime Governance for Autonomous AI Systems: The Aegis Architecture for Verifiable Policy Enforcement*. arXiv:2603.16938 [cs.CR], March 15, 2026.

Qin, X., Luan, S., See, J., Boukhers, Z., Yang, C., & Li, Z. (2026). Harnessing Embodied Agents: Runtime Governance for Policy-Constrained Execution. arXiv:2604.07833 [cs.RO], v2, April 10, 2026.

Kaul, A., Lan, Q., & Gupta, P. (2026). AgentBound: Verifiable Behavioral Governance for Autonomous AI Agents. arXiv:2606.30970 [cs.AI], June 2026.

End of Bibliography — Version 2.0.